7-2 · 텍스트 데이터 Chunking · 이론 1H

RAG 파이프라인과 Chunking 개념

RAG 여섯 단계, 그리고 조각 크기의 맞바꿈

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① RAG 여섯 단계를 순서대로 말하고 Chunking 의 위치를 짚는다

② 앞 네 단계는 미리, 뒤 두 단계는 질문마다 하는 것을 구별한다

③ 문서를 자르는 이유 네 가지를 설명한다

④ chunk_size 와 chunk_overlap 의 맞바꿈을 설명한다

다음 1.5H 실습에서 문서를 직접 읽고 잘라 경계를 눈으로 확인합니다

1. RAG 여섯 단계

오픈북 시험 — 찾아서(검색) 보면서 쓴다(생성)

Load	Chunk	Embed	Store	Retrieve	Generate
문서 읽기 미리	자르기 미리	벡터로 미리	벡터DB 저장 미리	찾기 질문마다	답 만들기 질문마다

앞 넷은 미리 해 두는 색인 작업 · 뒤 둘은 질문이 들어올 때마다 하는 일

이번 주 5일이 이 순서 그대로 — 책을 통째로 펼칠 수 없으니 미리 「찾기 좋은 크기」로 잘라 둔다 = Chunking

1. RAG 여섯 단계

단계마다 언제 하고, 이번 주 어느 날에 배우나

단계	언제	이번 주 어느 날
Load — 문서 읽기	미리	오늘
Chunk — 자르기	미리	오늘
Embed — 벡터로	미리	7-3
Store — 벡터DB 저장	미리	7-3 · 7-4
Retrieve — 찾기	질문마다	7-3 · 7-5
Generate — 답 만들기	질문마다	7-5

7-1 도해에서 위아래 두 줄로 나뉘어 있던 것이 바로 「미리」와 「질문마다」의 구분

통째로는 안 되는 이유 네 가지

01

컨텍스트 창 한계

한 번에 읽는 양에
한계가 있다

02

검색 정확도

통째로 한 벡터면
세부 내용이 흐려짐

03

비용

글이 길수록
토큰과 비용이 는다

04

근거 표시

조각 단위 출처로
어느 문서 몇 쪽

> **비용은 7-1 에서 본 그대로 — 넣는 글이 길수록 토큰이 늘고, 그래서 찾기 좋은 크기로 미리 잘라 둔다**

3. 조각 크기의 맞바꿈

chunk_size — 작으면 정확하고, 크면 문맥이 산다

chunk_size 가	장점	단점
작으면 (100~300)	검색이 정확해짐 — 딱 그 내용만	문맥이 끊겨 답이 부실해짐
크면 (1000~2000)	문맥이 살아 있음	관련 없는 내용이 딸려 와 흐려짐
적당히 (500~1000)	대체로 무난한 출발점	문서 성격에 따라 조정 필요

우리 문서로 재 보고 정하는 것이 유일한 방법 — 다음 실습 1H 에서 실측

「큰 게 좋다」도 「작은 게 좋다」도 아니다 — 조항이 독립적인 계약서는 작게, 서사가 이어지는 문서는 크게

3. 조각 크기의 맞바꿈

두 파라미터 — chunk_size 와 chunk_overlap

1. **chunk_size — 한 조각의 최대 글자 수**
   너무 크면 관련 없는 내용까지 딸려 오고, 너무 작으면 문맥이 끊긴다

2. **chunk_overlap — 이웃 조각이 겹치는 글자 수**
   필름을 이어 붙일 때 끝을 조금 겹치듯 — 문장 중간 절단의 피해를 줄인다

3. **overlap 은 보통 chunk_size 의 10~20%**
   정답은 문서마다 다르다 — 그래서 오늘 실측으로 비교한다

무난한 출발점

500~1000자

겹침 10~20%

4. 어디서 자를 것인가

separators 우선순위 — 자연스러운 경계부터

① 문단 \n\n	② 줄 \n	③ 공백	④ 글자
chunk_size 안이면 성공	안 되면 줄 경계로	안 되면 단어 경계로	그래도 안 되면 최후 수단

문서 형식에 맞게 바꾼다 — Markdown 은 "\n## " 를, 코드는 "\nclass " · "\ndef " 를 맨 앞에

Recursive 분할기는 문단 → 줄 → 공백 → 글자 순으로, 되도록 자연스러운 경계에서 자르려고 시도한다

Document — 본문과 출처를 함께 든다

page_content — 본문

조각의 글 자체 — 검색과 생성에 쓰인다

- "ETL은 추출, 변환, 적재의 순서로 …"
- 자른 뒤에는 조각마다 나뉘어 담긴다
- chunk_size 는 이 글자 수의 상한

VS

metadata — 출처 정보

source · page 등 — 답변 근거 표시에 쓴다

- {"source": "notes.txt", "page": 3}
- 자른 뒤에도 각 조각에 따라온다
- 답변에 「notes.txt 3쪽」이라고 밝힐 수 있다

정리

이 1시간에서 꼭 가져갈 것

1. RAG 는 Load 부터 Generate 까지 여섯 단계로 이어진다
2. 앞 네 단계는 미리(색인), 뒤 두 단계는 질문마다 한다
3. 자르는 이유는 컨텍스트 한계·검색 정확도·비용·출처 표시 넷
4. 조각이 작으면 정확하지만 문맥이 끊기고, 크면 반대다
5. overlap 은 문장 중간 절단의 피해를 줄인다
6. 메타데이터를 끝까지 들고 가야 출처를 밝힐 수 있다

7-2 · 이론 1H 마무리

다음은 Document Loader 와 분할 실습 (1.5H)

notes.txt 를 만들어 TextLoader 로 읽고, 200자 조각으로 잘라 경계와 겹침을 눈으로 확인합니다

- ✓ **TextLoader 로드와 metadata 확인**
- ✓ **일부러 cp949 로 읽어 오류 보기**
- ✓ **chunk_size 200 · overlap 30 으로 분할**
- ✓ **Recursive 와 Character 분할기 비교**

준비할 것

7-1 의 가상환경과 pip 설치 세 패키지